

```
from tensorflow.keras.datasets import fashion_mnist
```

```
%pip install numpy
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
```

```
(train_x, train_y), (test_x, test_y) = fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz  
29515/29515 ————— 0s 0us/step
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz  
26421880/26421880 ————— 0s 0us/step
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz  
5148/5148 ————— 0s 0us/step
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz  
4422102/4422102 ————— 0s 0us/step
```

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Flatten, MaxPooling2D, Conv2D
```

```
model = Sequential()
```

```
model.add(Conv2D(filters=64, kernel_size=(3,3), activation='relu', input_shape=(28, 28, 1)))
```

```
# Adding maxpooling layer to get max value within a matrix  
model.add(MaxPooling2D(pool_size=(2,2)))
```

```
model.add(Flatten())  
model.add(Dense(128, activation = "relu"))  
model.add(Dense(10, activation = "softmax"))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input`  
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 64)	640
max_pooling2d (MaxPooling2D)	(None, 13, 13, 64)	0
flatten (Flatten)	(None, 10816)	0
dense (Dense)	(None, 128)	1,384,576
dense_1 (Dense)	(None, 10)	1,290

Total params: 1,386,506 (5.29 MB)

Trainable params: 1,386,506 (5.29 MB)

Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer = 'adam', loss = 'sparse_categorical_crossentropy', metrics = ['accuracy'])
```

```
import numpy as np  
model.fit(train_x.astype(np.float32), train_y.astype(np.float32), epochs = 5, validation_split = 0.2)
```

```
Epoch 1/5  
1500/1500 ————— 65s 42ms/step - accuracy: 0.8468 - loss: 1.0923 - val_accuracy: 0.8764 - val_loss: 0.3626  
Epoch 2/5  
1500/1500 ————— 80s 41ms/step - accuracy: 0.8978 - loss: 0.2876 - val_accuracy: 0.8907 - val_loss: 0.3288  
Epoch 3/5  
1500/1500 ————— 60s 40ms/step - accuracy: 0.9105 - loss: 0.2461 - val_accuracy: 0.8918 - val_loss: 0.3165  
Epoch 4/5  
1500/1500 ————— 63s 42ms/step - accuracy: 0.9174 - loss: 0.2240 - val_accuracy: 0.8898 - val_loss: 0.3330  
Epoch 5/5  
1500/1500 ————— 59s 40ms/step - accuracy: 0.9245 - loss: 0.2052 - val_accuracy: 0.8893 - val_loss: 0.3360  
<keras.src.callbacks.history.History at 0x7d77af21c0e0>
```

```
loss, acc = model.evaluate(test_x, test_y)
```

```
313/313 ————— 4s 12ms/step - accuracy: 0.8851 - loss: 0.3493
```

```
labels = ['t_shirt', 'trouser', 'pullover', 'dress', 'coat', 'sandal', 'shirt', 'sneaker', 'bag', 'ankle_boots']
```

```
predictions = model.predict(test_x[:1])
```

1/1 ————— 0s 88ms/step

```
import numpy as np
```

```
label = labels[np.argmax(predictions)]
```

```
import matplotlib.pyplot as plt
print(label)
plt.imshow(test_x[:1][0])
plt.show
```

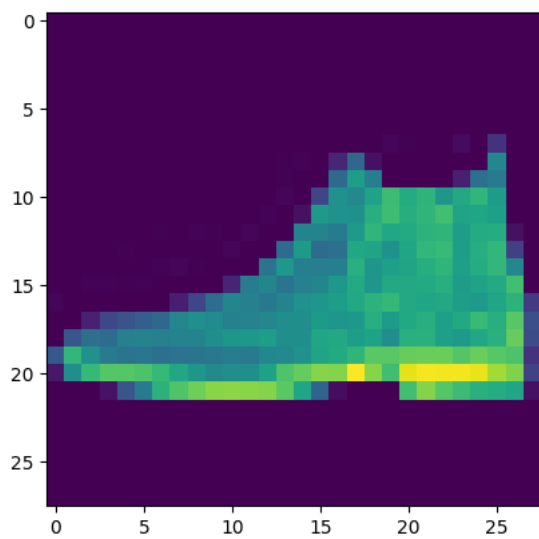
ankle_boots

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

</usr/local/lib/python3.12/dist-packages/matplotlib/pyplot.py>
Display all open figures.

Parameters

block : bool, optional



Start coding or [generate](#) with AI.